SOLID

Pengertian

Pada modul-modul sebelumnya kita sudah belajar membuat kelas, mewariskan atribut dan method, serta memakai polimorfisme dan abstraksi. Semakin banyak fitur yang ditambahkan, semakin besar kode yang harus dijaga. Tanpa aturan desain, satu perubahan kecil bisa merusak bagian lain yang terlihat tidak berhubungan.

SOLID adalah singkatan dari lima prinsip desain berorientasi objek yang dipopulerkan oleh Robert C. Martin. Tujuannya membuat kode lebih mudah dipahami, diubah, diuji, dan dikembangkan.

Huruf	Prinsip	Inti
S	Single Responsibility Principle	Satu kelas, satu alasan untuk berubah
O	Open/Closed Principle	Terbuka untuk ditambah, tertutup untuk diubah
L	Liskov Substitution Principle	Subclass bisa menggantikan parent tanpa merusak program
I	Interface Segregation Principle	Kelas tidak dipaksa memiliki method yang tidak dipakai
D	Dependency Inversion Principle	Bergantung pada abstraksi, bukan pada kelas konkret

Seluruh contoh di modul ini memakai studi kasus game MOBA, yaitu hero, minion, turret, dan arena pertarungan. Kelas Hero dari modul Inheritance dan Polimorfisme dibuat lebih sederhana supaya contoh kodenya pendek dan fokus pada prinsipnya.

Single Responsibility Principle (SRP)

Sebuah kelas sebaiknya hanya punya satu tanggung jawab, atau dengan kata lain hanya punya **satu alasan untuk berubah**. Kalau sebuah kelas berubah karena urusan tampilan,

berubah lagi karena urusan penyimpanan, lalu berubah lagi karena aturan pertarungan, kelas itu memegang terlalu banyak tanggung jawab.

Contoh Pelanggaran

Kelas Hero di bawah mengurus tiga hal sekaligus: aturan pertarungan, tampilan ke layar, dan penyimpanan ke file.

```
class Hero:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def serang(self, target):          # tugas 1: pertarungan
        target.health -= 10

    def tampilkan_status(self):        # tugas 2: tampilan
        print(f"{self.name} | HP {self.health}")

    def simpan(self, nama_file):       # tugas 3: penyimpanan
        with open(nama_file, "a") as f:
            f.write(f"{self.name},{self.health}\n")
```

Masalah dari kode di atas:

- Mengganti format tampilan, misalnya menambah bar HP, memaksa kita membuka kelas Hero.
- Mengganti tempat penyimpanan dari file ke database juga memaksa kita membuka kelas Hero.
- Untuk menguji serang(), kita ikut membawa kode file dan print yang tidak berhubungan.

Penerapan SRP

Pisahkan setiap tanggung jawab ke kelasnya sendiri.

```
class Hero:                            # hanya pertarungan
    def __init__(self, name, health):
```

```
        self.name = name
        self.health = health

    def serang(self, target):
        target.health -= 10


class TampilanHero:                    # hanya tampilan
    def tampilkan(self, hero):
        print(f"{hero.name} | HP {hero.health}")


class PenyimpanHero:                   # hanya penyimpanan
    def simpan(self, hero, nama_file):
        with open(nama_file, "a") as f:
            f.write(f"{hero.name},{hero.health}\n")


layla = Hero("Layla", 70)
Hero("Balmond", 120).serang(layla)
TampilanHero().tampilkan(layla)
PenyimpanHero().simpan(layla, "data_hero.txt")
```

Output:

```
Layla | HP 60
```

Sekarang perubahan tampilan hanya menyentuh TampilanHero, perubahan penyimpanan hanya menyentuh PenyimpanHero, dan Hero tetap fokus pada aturan pertarungan.

Open/Closed Principle (OCP)

Kelas harus **terbuka untuk ditambah** (open for extension), tetapi **tertutup untuk diubah** (closed for modification). Artinya, fitur baru ditambahkan dengan membuat kelas baru, bukan dengan membongkar kode lama yang sudah berjalan dan sudah diuji.

Contoh Pelanggaran

Setiap role dibedakan dengan string, lalu dihitung dengan rantai if dan elif.

```
def hitung_damage(role, attack):
    if role == "fighter":
        return attack + 5
    elif role == "mage":
        return attack * 2
    # role baru = ubah fungsi ini lagi
    return 0


print(hitung_damage("fighter", 10))
print(hitung_damage("mage", 10))
print(hitung_damage("assassin", 10))
```

Output:

```
15
20
0
```

Perhatikan baris terakhir. Role assassin mendapat damage 0 karena belum ditulis di hitung_damage(). Program tidak error, tetapi hasilnya salah. Setiap ada role baru, fungsi hitung_damage() harus diubah lagi dan rantai if terus memanjang.

Penerapan OCP

Gunakan abstraksi dan polimorfisme dari modul sebelumnya. Setiap role menjadi subclass yang menulis hitung_damage() miliknya sendiri, sehingga rantai if dan elif tidak diperlukan lagi.

```
class Hero:
    def __init__(self, attack):
        self.attack = attack

    def hitung_damage(self):
        return self.attack


class Fighter(Hero):
    def hitung_damage(self):
        return self.attack + 5
```

```
class Mage(Hero):
    def hitung_damage(self):
        return self.attack * 2


class Assassin(Hero):                    # role baru: cukup tambah kelas
    def hitung_damage(self):
        return self.attack * 3


print(Fighter(10).hitung_damage())
print(Mage(10).hitung_damage())
print(Assassin(10).hitung_damage())
```

Output:

```
15
20
30
```

Untuk menambah Assassin kita hanya membuat satu kelas baru. Kelas Hero, Fighter, dan Mage tidak disentuh sama sekali.

Liskov Substitution Principle (LSP)

Objek subclass harus bisa dipakai di mana pun objek parent-nya dipakai, tanpa membuat program error atau berperilaku aneh. Subclass boleh menambah kemampuan, tetapi **tidak boleh mengingkari janji** yang sudah dibuat parent.

Contoh Pelanggaran

Kelas Unit berjanji bahwa semua unit bisa bergerak. Turret adalah unit, tetapi tidak bisa bergerak, sehingga method bergerak() ditimpa dengan error.

```
class Unit:
    def bergerak(self):
```

```
        print("Unit bergerak")


class Minion(Unit):
    pass


class Turret(Unit):
    def bergerak(self):                # melanggar janji parent
        raise Exception("Turret tidak bisa bergerak!")


def maju(unit: Unit):
    unit.bergerak()


maju(Minion())
maju(Turret())
```

Output:

```
Unit bergerak
Exception: Turret tidak bisa bergerak!
```

Fungsi maju() menerima Unit, dan Turret memang Unit. Namun begitu Turret dikirim, program berhenti. Penyebabnya bukan fungsi maju(), melainkan hierarki kelas yang salah: Unit menjanjikan sesuatu yang tidak bisa ditepati semua anaknya.

Penerapan LSP

Pindahkan bergerak() ke kelas yang memang bisa menepatinya. Unit hanya berisi yang dimiliki semua unit, UnitBergerak menambahkan kemampuan bergerak, dan Turret tidak lagi berjanji bisa bergerak.

```
class Unit:
    pass                               # hanya yang dimiliki semua unit


class UnitBergerak(Unit):
    def bergerak(self):
```

```
        print("Unit bergerak")


class Minion(UnitBergerak):
    pass


class Turret(Unit):                    # tidak berjanji bisa bergerak
    def tembak(self):
        print("Turret menembak")


def maju(unit: UnitBergerak):
    unit.bergerak()


maju(Minion())
Turret().tembak()
```

Output:

```
Unit bergerak
Turret menembak
```

Sekarang setiap objek UnitBergerak pasti bisa bergerak. Type hint UnitBergerak pada maju() juga memberi tahu pembaca bahwa Turret tidak boleh dikirim ke fungsi ini.

Tanda pelanggaran LSP yang perlu diwaspadai:

- Subclass menimpa method parent hanya untuk melempar error.
- Subclass mengubah tipe nilai kembalian, misalnya parent mengembalikan angka tetapi subclass mengembalikan teks.
- Ada pengecekan isinstance() untuk menghindari subclass tertentu.

Interface Segregation Principle (ISP)

Kelas tidak boleh dipaksa memiliki method yang tidak dipakainya. Interface yang terlalu besar, yang di Python dibuat dengan Abstract Base Class, sebaiknya **dipecah menjadi beberapa interface kecil** yang lebih spesifik.

Contoh Pelanggaran

Interface KemampuanHero mewajibkan semua hero bisa menyerang dan heal. Fighter tidak bisa heal, tetapi tetap dipaksa menulis method itu.

```
from abc import ABC, abstractmethod

class KemampuanHero(ABC):
    @abstractmethod
    def serang(self):
        pass

    @abstractmethod
    def heal(self):
        pass


class Fighter(KemampuanHero):
    def serang(self):
        print("Fighter menyerang")

    def heal(self):                    # dipaksa ada, padahal tidak bisa
        raise NotImplementedError("Fighter tidak bisa heal")


Fighter().serang()
Fighter().heal()
```

Output:

```
Fighter menyerang
NotImplementedError: Fighter tidak bisa heal
```

Method heal() di Fighter hanya ada untuk memenuhi interface. Dari luar, pemanggil tidak bisa tahu bahwa method itu tidak berfungsi, dan baru tahu saat program error.

Penerapan ISP

Pecah kemampuan menjadi interface kecil. Setiap kelas hanya mewarisi kemampuan yang benar-benar dimilikinya, memanfaatkan multiple inheritance dari modul Inheritance.

```python
from abc import ABC, abstractmethod

class BisaMenyerang(ABC):
    @abstractmethod
    def serang(self):
        pass


class BisaHeal(ABC):
    @abstractmethod
    def heal(self):
        pass


class Fighter(BisaMenyerang):
    def serang(self):
        print("Fighter menyerang")


class Support(BisaMenyerang, BisaHeal):
    def serang(self):
        print("Support menyerang")

    def heal(self):
        print("Support memulihkan rekan")


Fighter().serang()
Support().heal()
print(isinstance(Fighter(), BisaHeal))
```

Output:

```
Fighter menyerang
Support memulihkan rekan
False
```

Fighter tidak lagi punya method palsu. Hasil isinstance(Fighter(), BisaHeal) bernilai False karena Fighter memang tidak bisa heal.

Dependency Inversion Principle (DIP)

Kelas tingkat tinggi tidak boleh bergantung langsung pada kelas tingkat rendah. Keduanya harus **bergantung pada abstraksi**. Dalam praktiknya, objek yang dibutuhkan sebuah kelas tidak dibuat di dalam kelas itu, tetapi dikirim dari luar. Teknik ini disebut dependency injection.

Contoh Pelanggaran

Arena membuat PapanSkorKonsol sendiri di dalam constructor.

```
class PapanSkorKonsol:
    def catat(self, pesan):
        print(f"[KONSOL] {pesan}")


class Arena:
    def __init__(self):
        self.papan_skor = PapanSkorKonsol()   # hardcoded

    def adu(self, a, b):
        self.papan_skor.catat(f"{a} vs {b}")


Arena().adu("Balmond", "Layla")
```

Output:

```
[KONSOL] Balmond vs Layla
```

Kalau turnamen ingin mencatat riwayat ke file, kelas Arena harus diubah. Arena juga sulit diuji tanpa ikut mencetak ke layar, karena papan skornya tidak bisa diganti.

Penerapan DIP

Buat abstraksi PapanSkor, lalu kirim objek papan skor ke Arena saat Arena dibuat.

```
from abc import ABC, abstractmethod
```

```python
class PapanSkor(ABC):                          # abstraksi
    @abstractmethod
    def catat(self, pesan):
        pass

class PapanSkorKonsol(PapanSkor):
    def catat(self, pesan):
        print(f"[KONSOL] {pesan}")

class PapanSkorFile(PapanSkor):
    def catat(self, pesan):
        with open("riwayat.txt", "a") as f:
            f.write(pesan + "\n")
        print("[FILE] tersimpan di riwayat.txt")

class Arena:
    def __init__(self, papan_skor: PapanSkor):  # disuntikkan dari luar
        self.papan_skor = papan_skor
    def adu(self, a, b):
        self.papan_skor.catat(f"{a} vs {b}")

Arena(PapanSkorKonsol()).adu("Balmond", "Layla")
Arena(PapanSkorFile()).adu("Layla", "Balmond")
```

Output:

```
[KONSOL] Balmond vs Layla
[FILE] tersimpan di riwayat.txt
```

File riwayat.txt berisi baris "Layla vs Balmond". Kelas Arena yang sama dipakai dengan dua papan skor berbeda tanpa diubah. Untuk papan skor jenis baru, misalnya yang mengirim pesan ke Discord, cukup buat subclass PapanSkor baru.

Studi Kasus: Sistem Pertarungan Mini

Kelima prinsip biasanya dipakai bersamaan. Kode berikut menggabungkan contoh-contoh di atas menjadi satu sistem pertarungan kecil.

```python
from abc import ABC, abstractmethod

class BisaHeal(ABC):                            # ISP: kemampuan terpisah
    @abstractmethod
    def heal(self, target):
        pass

class Hero(ABC):                                # OCP + LSP
    def __init__(self, name, health, attack):
        self.name = name
        self.health = health
        self.attack = attack

    @abstractmethod
    def hitung_damage(self):
        pass

class Fighter(Hero):
    def hitung_damage(self):
        return self.attack + 5

class Support(Hero, BisaHeal):
    def hitung_damage(self):
        return self.attack

    def heal(self, target):
        target.health += 20

class PapanSkorKonsol:                          # SRP: hanya tampilan
    def catat(self, pesan):
        print(f"[ARENA] {pesan}")

class Arena:
    def __init__(self, papan_skor):             # DIP: disuntikkan
        self.papan_skor = papan_skor

    def adu(self, penyerang, target):
        target.health -= penyerang.hitung_damage()
        self.papan_skor.catat(f"{target.name} HP {target.health}")

balmond = Fighter("Balmond", 120, 15)
estes = Support("Estes", 90, 10)

arena = Arena(PapanSkorKonsol())
```

```
arena.adu(balmond, estes)
arena.adu(estes, balmond)
```

Output:

```
[ARENA] Estes HP 70
[ARENA] Balmond HP 110
```

Penerapan tiap prinsip pada studi kasus di atas:

Prinsip	Penerapan di studi kasus
SRP	Hero hanya mengurus data dan damage, PapanSkorKonsol hanya menampilkan pesan
OCP	Fighter dan Support menulis hitung_damage() sendiri. Role baru cukup ditambah dengan kelas baru
LSP	Semua subclass Hero menepati kontrak hitung_damage(), sehingga Arena bisa memakai hero apa pun
ISP	Kemampuan heal dipisah ke BisaHeal dan hanya diwarisi Support
DIP	Arena menerima papan skor dari luar, tidak membuat PapanSkorKonsol sendiri

Cara Mendeteksi Pelanggaran SOLID

Prinsip	Tanda pelanggaran	Cara memperbaiki
SRP	Satu kelas punya method dari urusan berbeda, seperti hitung, tampilkan, dan simpan	Pisahkan setiap urusan ke kelasnya sendiri
OCP	Rantai if dan elif yang mengecek tipe atau role dalam bentuk string	Gunakan subclass dan polimorfisme

LSP	Subclass menimpa method parent hanya untuk melempar error	Perbaiki hierarki, pindahkan method ke kelas yang tepat
ISP	Subclass menulis method yang langsung melempar NotImplementedError	Pecah interface menjadi beberapa yang lebih kecil
DIP	Objek dibuat langsung di dalam __init__, misalnya self.papan_skor = PapanSkorKonsol()	Terima objek lewat parameter dan bergantung pada abstraksi

Checklist Sebelum Menulis Kelas

1. Apakah kelas ini punya lebih dari satu alasan untuk berubah? (SRP)
2. Apakah menambah jenis baru memaksa saya mengubah kode lama? (OCP)
3. Apakah setiap subclass bisa dipakai di tempat parent-nya tanpa error? (LSP)
4. Apakah ada kelas yang dipaksa menulis method yang tidak dipakainya? (ISP)
5. Apakah kelas ini membuat sendiri objek yang dibutuhkannya? (DIP)

SOLID Bukan Aturan Mutlak

SOLID adalah panduan, bukan kewajiban di setiap baris kode. Program kecil yang hanya dipakai sekali tidak perlu dipecah menjadi puluhan kelas. Memecah kode secara berlebihan justru membuat program sulit dibaca, kondisi yang disebut over-engineering. Terapkan SOLID ketika kode mulai sering berubah, mulai dikerjakan bersama tim, atau mulai sulit diuji.